

Punto 5.11 — Testing con EF Core: InMemory y SQLite in-memory

Práctica

Ejercicio: Crear el proyecto de pruebas `AceriaData.Tests` con `xUnit`. Configurar los proveedores `InMemory` y `SQLite` en memoria. Escribir tests que verifiquen el comportamiento del repositorio de órdenes y de la unidad de trabajo. Comparar el comportamiento de ambos proveedores.

Contexto del proyecto: En el punto 5.10 se estudió el logging y el diagnóstico, incluyendo la integración con `Serilog` y el logging estructurado. En este punto se profundiza en el testing con `EF Core`, que es esencial para garantizar la calidad de la capa de persistencia. Esta técnica se usará en el punto 5.12 para las buenas prácticas y anti-patrones.

Paso 1: Crear el proyecto de pruebas

```
cd AceriaData
dotnet new xunit -n AceriaData.Tests -f net8.0 -o tests/AceriaData.Tests
dotnet sln add tests/AceriaData.Tests/AceriaData.Tests.csproj
```

`dotnet new xunit` → crea un proyecto de pruebas con `xUnit`. `-n AceriaData.Tests` → asigna el nombre al proyecto. `-f net8.0` → especifica `.NET 8`. `-o tests/AceriaData.Tests` → especifica la carpeta de salida. `dotnet sln add` → añade el proyecto a la solución.

Resultado esperado: se crea el proyecto `AceriaData.Tests` en la carpeta `tests`.

Error común: si la carpeta `tests` no existe, el comando la crea. Si el proyecto ya existe, el comando falla.

Paso 2: Añadir las referencias a los proyectos

```
dotnet add tests/AceriaData.Tests/AceriaData.Tests.csproj reference src/AceriaData.Domain/
AceriaData.Domain.csproj
dotnet add tests/AceriaData.Tests/AceriaData.Tests.csproj reference src/AceriaData.Application/
AceriaData.Application.csproj
dotnet add tests/AceriaData.Tests/AceriaData.Tests.csproj reference src/AceriaData.Infrastructure/
AceriaData.Infrastructure.csproj
```

`dotnet add reference` → añade una referencia entre proyectos.

Resultado esperado: el proyecto de pruebas referencia a los tres proyectos.

Error común: si la ruta es incorrecta, el comando falla. Se debe verificar la ruta.

Paso 3: Añadir los paquetes de EF Core para pruebas

```
dotnet add tests/AceriaData.Tests/AceriaData.Tests.csproj package Microsoft.EntityFrameworkCore.InMemory
dotnet add tests/AceriaData.Tests/AceriaData.Tests.csproj package Microsoft.EntityFrameworkCore.Sqlite
```

`dotnet add package` → añade una referencia a un paquete `NuGet`.

Resultado esperado: el proyecto de pruebas referencia a los dos proveedores.

Error común: si se olvida alguno de los paquetes, las pruebas no compilan.

Paso 4: Crear la clase base de pruebas

Crear el archivo `tests/AceriaData.Tests/Base/TestBase.cs`:

```

using AceriaData.Infrastructure.Persistence;
using Microsoft.Data.Sqlite;
using Microsoft.EntityFrameworkCore;

namespace AceriaData.Tests.Base;

public abstract class TestBase : IDisposable
{
    protected readonly AceriaDbContext Context;
    private readonly SqliteConnection _connection;

    protected TestBase()
    {
        _connection = new SqliteConnection("Data Source=:memory:");
        _connection.Open();

        var options = new DbContextOptionsBuilder<AceriaDbContext>()
            .UseSqlite(_connection)
            .Options;

        Context = new AceriaDbContext(options);
        Context.Database.EnsureCreated();
    }

    public void Dispose()
    {
        Context.Dispose();
        _connection.Dispose();
    }
}

```

Línea 1: using AceriaData.Infrastructure.Persistence; → importa el DbContext.

Línea 2: using Microsoft.Data.Sqlite; → importa SQLite.

Línea 3: using Microsoft.EntityFrameworkCore; → importa EF Core.

Línea 5: namespace AceriaData.Tests.Base; → declara el espacio de nombres.

Línea 7: public abstract class TestBase : IDisposable → declara la clase base.

Línea 9: protected readonly AceriaDbContext Context; → campo del contexto.

Línea 10: private readonly SqliteConnection _connection; → campo de la conexión.

Línea 12: protected TestBase() → constructor.

Línea 14: _connection = new SqliteConnection("Data Source=:memory:"); → crea la conexión en memoria.

Línea 15: _connection.Open(); → abre la conexión.

Línea 17: var options = new DbContextOptionsBuilder() → crea el constructor de opciones.

Línea 18: .UseSqlite(_connection) → configura SQLite.

Línea 19: .Options; → obtiene las opciones.

Línea 21: Context = new AceriaDbContext(options); → crea el contexto.

Línea 22: Context.Database.EnsureCreated(); → crea la base de datos.

Línea 25: public void Dispose() → declara el método Dispose.

Línea 27: Context.Dispose(); → libera el contexto.

Línea 28: _connection.Dispose(); → libera la conexión.

Error común: si se olvida abrir la conexión, la base de datos en memoria no existe y las pruebas fallan.

Paso 5: Crear tests del repositorio de órdenes

Crear el archivo tests/AceriaData.Tests/Repositories/OrdenRepositorioTests.cs:

```
using AceriaData.Domain.Entities;
using AceriaData.Infrastructure.Repositories;
using AceriaData.Tests.Base;
using Microsoft.Extensions.Logging.Abstractions;

namespace AceriaData.Tests.Repositories;

public class OrdenRepositorioTests : TestBase
{
    private readonly OrdenRepositorio _repositorio;

    public OrdenRepositorioTests()
    {
        _repositorio = new OrdenRepositorio(Context, NullLogger<OrdenRepositorio>.Instance);
    }

    [Fact]
    public void ObtenerPorId_ConIdExistente_DevuelveLaOrden()
    {
        var orden = new OrdenFabricacion { NumeroOrden = "0F-001", Cliente = "Constructora del Norte",
        Estado = "Pendiente", FechaCreacion = DateTime.Now };
        Context.OrdenesFabricacion.Add(orden);
        Context.SaveChanges();

        var recuperada = _repositorio.ObtenerPorId(orden.Id);

        Assert.NotNull(recuperada);
        Assert.Equal("0F-001", recuperada.NumeroOrden);
    }

    [Fact]
    public void ObtenerPorId_ConIdInexistente_DevuelveNull()
    {
        var recuperada = _repositorio.ObtenerPorId(99999);

        Assert.Null(recuperada);
    }

    [Fact]
    public void ObtenerPendientes_ConVariasOrdenes_DevuelveSoloLasPendientes()
    {
        Context.OrdenesFabricacion.AddRange(
            new OrdenFabricacion { NumeroOrden = "0F-001", Cliente = "A", Estado = "Pendiente",
            FechaCreacion = DateTime.Now },
            new OrdenFabricacion { NumeroOrden = "0F-002", Cliente = "B", Estado = "EnProceso",
            FechaCreacion = DateTime.Now },
            new OrdenFabricacion { NumeroOrden = "0F-003", Cliente = "C", Estado = "Pendiente",
            FechaCreacion = DateTime.Now });
        Context.SaveChanges();

        var pendientes = _repositorio.ObtenerPendientes();
        Assert.Equal(2, pendientes.Count);
    }

    [Fact]
    public void ContarOrdenes_ConVariasOrdenes_DevuelveElTotal()
    {
        Context.OrdenesFabricacion.AddRange(
            new OrdenFabricacion { NumeroOrden = "0F-001", Cliente = "A", Estado = "Pendiente",
            FechaCreacion = DateTime.Now },
            new OrdenFabricacion { NumeroOrden = "0F-002", Cliente = "B", Estado = "EnProceso",
            FechaCreacion = DateTime.Now });
        Context.SaveChanges();

        var total = _repositorio.ContarOrdenes();

        Assert.Equal(2, total);
    }
}
```

```

[Fact]
public void ExisteAlgunaOrden_SinOrdenes_DevuelveFalse()
{
    var existe = _repositorio.ExisteAlgunaOrden();

    Assert.False(existe);
}
}

```

Línea 1: using AceriaData.Domain.Entities; → importa las entidades.

Línea 2: using AceriaData.Infrastructure.Repositories; → importa el repositorio.

Línea 3: using AceriaData.Tests.Base; → importa la clase base.

Línea 4: using Microsoft.Extensions.Logging.Abstractions; → importa NullLogger.

Línea 6: namespace AceriaData.Tests.Repositories; → declara el espacio de nombres.

Línea 8: public class OrdenRepositorioTests : TestBase → declara la clase de tests.

Línea 10: private readonly OrdenRepositorio _repositorio; → campo del repositorio.

Línea 12: public OrdenRepositorioTests() → constructor.

Línea 14: _repositorio = new OrdenRepositorio(Context, NullLogger.Instance); → crea el repositorio con NullLogger.

Línea 17: [Fact] → atributo de test.

Línea 18: public void ObtenerPorId_ConIdExistente_DevuelveLaOrden() → declara el test.

Línea 20: var orden = new OrdenFabricacion { ... }; → crea la orden.

Línea 21: Context OrdenesFabricacion.Add(orden); → registra la orden.

Línea 22: Context.SaveChanges(); → guarda los cambios.

Línea 24: var recuperada = _repositorio.ObtenerPorId(orden.Id); → llama al método.

Línea 26: Assert.NotNull(recuperada); → comprueba que existe.

Línea 27: Assert.Equal("OF-001", recuperada.NumeroOrden); → comprueba el número.

Línea 30: [Fact] → atributo de test.

Línea 31: public void ObtenerPorId_ConIdInexistente_DevuelveNull() → declara el test.

Línea 33: var recuperada = _repositorio.ObtenerPorId(99999); → llama al método.

Línea 35: Assert.Null(recuperada); → comprueba que es nulo.

Línea 38: [Fact] → atributo de test.

Línea 39: public void ObtenerPendientes_ConVariasOrdenes_DevuelveSoloLasPendientes() → declara el test.

Línea 41: Context.OrdenesFabricacion.AddRange(→ añade varias órdenes.

Línea 45: Context.SaveChanges(); → guarda los cambios.

Línea 47: var pendientes = _repositorio.ObtenerPendientes(); → llama al método.

Línea 49: Assert.Equal(2, pendientes.Count); → comprueba el número.

Línea 52: [Fact] → atributo de test.

Línea 53: public void ContarOrdenes_ConVariasOrdenes_DevuelveElTotal() → declara el test.

Línea 55: Context.OrdenesFabricacion.AddRange(→ añade varias órdenes.

Línea 58: `Context.SaveChanges();` → guarda los cambios.

Línea 60: `var total = _repositorio.ContarOrdenes();` → llama al método.

Línea 62: `Assert.Equal(2, total);` → comprueba el total.

Línea 65: `[Fact]` → atributo de test.

Línea 66: `public void ExisteAlgunaOrden_SinOrdenes_DevuelveFalse()` → declara el test.

Línea 68: `var existe = _repositorio.ExisteAlgunaOrden();` → llama al método.

Línea 70: `Assert.False(existe);` → comprueba que es falso.

Error común: si el repositorio no acepta `ILogger` en el constructor, el código no compila. Se debe inyectar el logger o usar `NullLogger`.

| Paso 6: Crear tests de la unidad de trabajo

Crear el archivo `tests/AceriaData.Tests/Repositories/UnidadDeTrabajoTests.cs`:

```
using AceriaData.Domain.Entities;
using AceriaData.Infrastructure.Repositories;
using AceriaData.Tests.Base;

namespace AceriaData.Tests.Repositories;

public class UnidadDeTrabajoTests : TestBase
{
    [Fact]
    public void Guardar_ConOrdenYPlancha_InsertaAmbasEnUnaTransaccion()
    {
        using var unidad = new UnidadDeTrabajo(Context);

        var orden = new OrdenFabricacion
        {
            NumeroOrden = "0F-UOW-001",
            Cliente = "Constructora del Norte",
            Estado = "Pendiente",
            FechaCreacion = DateTime.Now
        };
        unidad Ordenes.Agregar(orden);

        var plancha = new PlanchaAcero
        {
            Orden = orden,
            Espesor = 10.5,
            Ancho = 1500,
            Largo = 3000,
            Peso = 370.5m,
            Activa = true
        };
        unidad Planchas.Agregar(plancha);

        var filas = unidad.Guardar();

        Assert.Equal(2, filas);
        Assert.NotEqual(0, orden.Id);
        Assert.NotEqual(0, plancha.Id);
    }

    [Fact]
    public void Guardar_SinCambios_DevuelveCero()
    {
        using var unidad = new UnidadDeTrabajo(Context);

        var filas = unidad.Guardar();

        Assert.Equal(0, filas);
    }
}
```

```

[Fact]
public void Guardar_ConDosRepositorios_CompartenElMismoContexto()
{
    using var unidad = new UnidadDeTrabajo(Context);

    var orden = new OrdenFabricacion { NumeroOrden = "OF-001", Cliente = "A", Estado = "Pendiente",
FechaCreacion = DateTime.Now };
    unidad Ordenes.Agregar(orden);
    unidad.Guardar();

    var plancha = new PlanchaAcero { OrdenId = orden.Id, Espesor = 10.5, Ancho = 1500, Largo =
3000, Peso = 370.5m, Activa = true };
    unidad.Planchas.Agregar(plancha);
    var filas = unidad.Guardar();

    Assert.Equal(1, filas);
}
}

```

Línea 1: using AceriaData.Domain.Entities; → importa las entidades.

Línea 2: using AceriaData.Infrastructure.Repositories; → importa la unidad de trabajo.

Línea 3: using AceriaData.Tests.Base; → importa la clase base.

Línea 5: namespace AceriaData.Tests.Repositories; → declara el espacio de nombres.

Línea 7: public class UnidadDeTrabajoTests : TestBase → declara la clase de tests.

Línea 9: [Fact] → atributo de test.

Línea 10: public void Guardar_ConOrdenYPlancha_InsertaAmbasEnUnaTransaccion() → declara el test.

Línea 12: using var unidad = new UnidadDeTrabajo(Context); → crea la unidad de trabajo.

Línea 14: var orden = new OrdenFabricacion → crea la orden.

Línea 22: unidad Ordenes.Agregar(orden); → agrega la orden.

Línea 24: var plancha = new PlanchaAcero → crea la plancha.

Línea 34: unidad.Planchas.Agregar(plancha); → agrega la plancha.

Línea 36: var filas = unidad.Guardar(); → guarda los cambios.

Línea 38: Assert.Equal(2, filas); → comprueba el número de filas.

Línea 39: Assert.NotEqual(0, orden.Id); → comprueba que el Id se ha generado.

Línea 40: Assert.NotEqual(0, plancha.Id); → comprueba que el Id se ha generado.

Línea 43: [Fact] → atributo de test.

Línea 44: public void Guardar_SinCambios_DevuelveCero() → declara el test.

Línea 46: using var unidad = new UnidadDeTrabajo(Context); → crea la unidad de trabajo.

Línea 48: var filas = unidad.Guardar(); → guarda los cambios.

Línea 50: Assert.Equal(0, filas); → comprueba que devuelve cero.

Línea 53: [Fact] → atributo de test.

Línea 54: public void Guardar_ConDosRepositorios_CompartenElMismoContexto() → declara el test.

Línea 56: using var unidad = new UnidadDeTrabajo(Context); → crea la unidad de trabajo.

Línea 58: var orden = new OrdenFabricacion { ... }; → crea la orden.

Línea 59: unidad Ordenes.Agregar(orden); → agrega la orden.

Línea 60: unidad.Guardar(); → guarda los cambios.

Línea 62: `var plancha = new PlanchaAcero { ... };` → crea la plancha.

Línea 63: `unidad.Planchas.Agregar(plancha);` → agrega la plancha.

Línea 64: `var filas = unidad.Guardar();` → guarda los cambios.

Línea 66: `Assert.Equal(1, filas);` → comprueba el número de filas.

Error común: si se crea una unidad de trabajo por repositorio, no comparten el mismo contexto y no se pueden guardar en una sola transacción. Se debe usar una sola unidad de trabajo.

Paso 7: Crear tests con InMemory

Crear el archivo `tests/AceriaData.Tests/InMemory/OrdenRepositorioInMemoryTests.cs`:

```
using AceriaData.Domain.Entities;
using AceriaData.Infrastructure.Persistence;
using AceriaData.Infrastructure.Repositories;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Logging.Abstractions;

namespace AceriaData.Tests.InMemory;

public class OrdenRepositorioInMemoryTests : IDisposable
{
    private readonly AceriaDbContext _context;
    private readonly OrdenRepositorio _repositorio;

    public OrdenRepositorioInMemoryTests()
    {
        var options = new DbContextOptionsBuilder<AceriaDbContext>()
            .UseInMemoryDatabase(Guid.NewGuid().ToString())
            .Options;

        _context = new AceriaDbContext(options);
        _repositorio = new OrdenRepositorio(_context, NullLogger<OrdenRepositorio>.Instance);
    }

    [Fact]
    public void InsertarOrden_ConDatosValidos_SeGuardaEnMemoria()
    {
        var orden = new OrdenFabricacion { NumeroOrden = "OF-001", Cliente = "Constructora del Norte",
        Estado = "Pendiente", FechaCreacion = DateTime.Now };
        _repositorio.Agregar(orden);
        _context.SaveChanges();

        var recuperada = _repositorio.ObtenerPorNumeroOrden("OF-001");

        Assert.NotNull(recuperada);
        Assert.Equal("Constructora del Norte", recuperada.Cliente);
    }

    [Fact]
    public void InsertarPlanchaConOrdenInvalida_NoLanzaExcepcion()
    {
        var plancha = new PlanchaAcero { OrdenId = 99999, Espesor = 10.5, Ancho = 1500, Largo = 3000,
        Peso = 370.5m, Activa = true };
        _context.PlanchasAcero.Add(plancha);

        var excepcion = Record.Exception(() => _context.SaveChanges());

        Assert.Null(excepcion);
    }

    public void Dispose()
    {
        _context.Dispose();
    }
}
```

Línea 1: `using AceriaData.Domain.Entities;` → importa las entidades.

Línea 2: `using AceriaData.Infrastructure.Persistence;` → importa el DbContext.

Línea 3: `using AceriaData.Infrastructure.Repositories;` → importa el repositorio.

Línea 4: `using Microsoft.EntityFrameworkCore;` → importa EF Core.

Línea 5: `using Microsoft.Extensions.Logging.Abstractions;` → importa NullLogger.

Línea 7: `namespace AceriaData.Tests.InMemory;` → declara el espacio de nombres.

Línea 9: `public class OrdenRepositorioInMemoryTests : IDisposable` → declara la clase de tests.

Línea 11: `private readonly AceriaDbContext _context;` → campo del contexto.

Línea 12: `private readonly OrdenRepositorio _repositorio;` → campo del repositorio.

Línea 14: `public OrdenRepositorioInMemoryTests()` → constructor.

Línea 16: `var options = new DbContextOptionsBuilder()` → crea el constructor de opciones.

Línea 17: `.UseInMemoryDatabase(Guid.NewGuid().ToString())` → configura InMemory con un nombre único.

Línea 18: `.Options;` → obtiene las opciones.

Línea 20: `_context = new AceriaDbContext(options);` → crea el contexto.

Línea 21: `_repositorio = new OrdenRepositorio(_context, NullLogger.Instance);` → crea el repositorio.

Línea 24: `[Fact]` → atributo de test.

Línea 25: `public void InsertarOrden_ConDatosValidos_SeGuardaEnMemoria()` → declara el test.

Línea 27: `var orden = new OrdenFabricacion { ... };` → crea la orden.

Línea 28: `_repositorio.Agregar(orden);` → agrega la orden.

Línea 29: `_context.SaveChanges();` → guarda los cambios.

Línea 31: `var recuperada = _repositorio.ObtenerPorNumeroOrden("OF-001");` → recupera la orden.

Línea 33: `Assert.NotNull(recuperada);` → comprueba que existe.

Línea 34: `Assert.Equal("Constructora del Norte", recuperada.Cliente);` → comprueba el cliente.

Línea 37: `[Fact]` → atributo de test.

Línea 38: `public void InsertarPlanchaConOrdenInvalida_NoLanzaExcepcion()` → declara el test.

Línea 40: `var plancha = new PlanchaAcero { OrdenId = 99999, ... };` → crea la plancha con clave foránea inválida.

Línea 41: `_context.PlanchasAcero.Add(plancha);` → registra la plancha.

Línea 43: `var excepcion = Record.Exception(() => _context.SaveChanges());` → ejecuta y captura la excepción.

Línea 45: `Assert.Null(excepcion);` → comprueba que no hay excepción.

Línea 48: `public void Dispose()` → declara el método Dispose.

Línea 50: `_context.Dispose();` → libera el contexto.

Error común: el test `InsertarPlanchaConOrdenInvalida_NoLanzaExcepcion` demuestra una limitación del proveedor InMemory. En SQLite en memoria, este test fallaría porque la restricción de integridad referencial se aplica.

Paso 8: Ejecutar los tests

```
dotnet test tests/AceriaData.Tests/AceriaData.Tests.csproj
```

dotnet test → ejecuta los tests del proyecto.

Resultado esperado: todos los tests pasan. El test InsertarPlanchaConOrdenInvalida_NoLanzaExcepcion pasa con InMemory pero fallaría con SQLite en memoria.

Error común: si algún test falla, se debe revisar el mensaje de error y corregir el código.

Paso 9: Crear tests con SQLite en memoria

Crear el archivo tests/AceriaData.Tests/Sqlite/OrdenRepositorioSqliteTests.cs:

```
using AceriaData.Domain.Entities;
using AceriaData.Infrastructure.Repositories;
using AceriaData.Tests.Base;

namespace AceriaData.Tests.Sqlite;

public class OrdenRepositorioSqliteTests : TestBase
{
    private readonly OrdenRepositorio _repositorio;

    public OrdenRepositorioSqliteTests()
    {
        _repositorio = new OrdenRepositorio(Context,
        Microsoft.Extensions.Logging.Abstractions.NullLogger<OrdenRepositorio>.Instance);
    }

    [Fact]
    public void InsertarOrden_ConDatosValidos_SeGuardaEnSqlite()
    {
        var orden = new OrdenFabricacion { NumeroOrden = "OF-001", Cliente = "Constructora del Norte",
        Estado = "Pendiente", FechaCreacion = DateTime.Now };
        _repositorio.Agregar(orden);
        Context.SaveChanges();

        var recuperada = _repositorio.ObtenerPorNumeroOrden("OF-001");

        Assert.NotNull(recuperada);
        Assert.Equal("Constructora del Norte", recuperada.Cliente);
    }

    [Fact]
    public void InsertarPlanchaConOrdenInvalida_LanzaExcepcion()
    {
        var plancha = new PlanchaAcero { OrdenId = 99999, Espesor = 10.5, Ancho = 1500, Largo = 3000,
        Peso = 370.5m, Activa = true };
        Context.PlanchasAcero.Add(plancha);

        var excepcion = Record.Exception(() => Context.SaveChanges());

        Assert.NotNull(excepcion);
    }

    [Fact]
    public void InsertarOrdenConNumeroDuplicado_LanzaExcepcion()
    {
        Context OrdenesFabricacion.Add(new OrdenFabricacion { NumeroOrden = "OF-001", Cliente = "A",
        Estado = "Pendiente", FechaCreacion = DateTime.Now });
        Context.SaveChanges();

        Context OrdenesFabricacion.Add(new OrdenFabricacion { NumeroOrden = "OF-001", Cliente = "B",
        Estado = "Pendiente", FechaCreacion = DateTime.Now });

        var excepcion = Record.Exception(() => Context.SaveChanges());
    }
}
```

```

        Assert.NotNull(excepcion);
    }
}

```

Línea 1: using AceriaData.Domain.Entities; → importa las entidades.

Línea 2: using AceriaData.Infrastructure.Repositories; → importa el repositorio.

Línea 3: using AceriaData.Tests.Base; → importa la clase base.

Línea 5: namespace AceriaData.Tests.Sqlite; → declara el espacio de nombres.

Línea 7: public class OrdenRepositorioSqliteTests : TestBase → declara la clase de tests.

Línea 9: private readonly OrdenRepositorio _repositorio; → campo del repositorio.

Línea 11: public OrdenRepositorioSqliteTests() → constructor.

Línea 13: _repositorio = new OrdenRepositorio(Context, ...); → crea el repositorio.

Línea 17: [Fact] → atributo de test.

Línea 18: public void InsertarOrden_ConDatosValidos_SeGuardaEnSqlite() → declara el test.

Línea 20: var orden = new OrdenFabricacion { ... }; → crea la orden.

Línea 21: _repositorio.Agregar(orden); → agrega la orden.

Línea 22: Context.SaveChanges(); → guarda los cambios.

Línea 24: var recuperada = _repositorio.ObtenerPorNumeroOrden("OF-001"); → recupera la orden.

Línea 26: Assert.NotNull(recuperada); → comprueba que existe.

Línea 27: Assert.Equal("Constructora del Norte", recuperada.Cliente); → comprueba el cliente.

Línea 30: [Fact] → atributo de test.

Línea 31: public void InsertarPlanchaConOrdenInvalida_LanzaExcepcion() → declara el test.

Línea 33: var plancha = new PlanchaAcero { OrdenId = 99999, ... }; → crea la plancha con clave foránea inválida.

Línea 34: Context.PlanchasAcero.Add(plancha); → registra la plancha.

Línea 36: var excepcion = Record.Exception(() => Context.SaveChanges()); → captura la excepción.

Línea 38: Assert.NotNull(excepcion); → comprueba que hay excepción.

Línea 41: [Fact] → atributo de test.

Línea 42: public void InsertarOrdenConNumeroDuplicado_LanzaExcepcion() → declara el test.

Línea 44: Context OrdenesFabricacion.Add(new OrdenFabricacion { ... }); → añade la primera orden.

Línea 45: Context.SaveChanges(); → guarda los cambios.

Línea 47: Context OrdenesFabricacion.Add(new OrdenFabricacion { ... }); → añade la segunda orden con el mismo número.

Línea 49: var excepcion = Record.Exception(() => Context.SaveChanges()); → captura la excepción.

Línea 51: Assert.NotNull(excepcion); → comprueba que hay excepción.

Error común: el test InsertarPlanchaConOrdenInvalida_LanzaExcepcion pasa con SQLite en memoria porque la restricción de integridad referencial se aplica. En InMemory, este test fallaría porque la restricción no se aplica.

Paso 10: Ejecutar todos los tests

```
dotnet test tests/AceriaData.Tests/AceriaData.Tests.csproj
```

dotnet test → ejecuta todos los tests.

Resultado esperado: todos los tests pasan. Los tests con SQLite en memoria aplican las restricciones de integridad referencial y de unicidad.

Paso 11: Analizar la salida

La salida del comando muestra información como la siguiente:

```
Passed! - Failed: 0, Passed: 10, Skipped: 0, Total: 10, Duration: 1 s
La primera línea muestra el resultado. Todos los tests pasan.
```

Observaciones: los tests con SQLite en memoria aplican las restricciones de integridad referencial y de unicidad. Los tests con InMemory no las aplican. La elección del proveedor depende del tipo de prueba.

Paso 12: Diagnosticar un error común

Modificar el test InsertarPlanchaConOrdenInvalida_NoLanzaExcepcion para ejecutarlo con SQLite en memoria:

```
[Fact]
public void InsertarPlanchaConOrdenInvalida_ConSqlite_LanzaExcepcion()
{
    var plancha = new PlanchaAcero { OrdenId = 99999, Espesor = 10.5, Ancho = 1500, Largo = 3000, Peso
= 370.5m, Activa = true };
    Context.PlanchasAcero.Add(plancha);

    var excepcion = Record.Exception(() => Context.SaveChanges());

    Assert.NotNull(excepcion);
}
```

Resultado esperado: con SQLite en memoria, el test pasa porque la restricción de integridad referencial se aplica. Con InMemory, el test fallaría.

Solución: usar SQLite en memoria para las pruebas que dependen de las restricciones de la base de datos. Usar InMemory para las pruebas que no dependen de las restricciones.

Resultado esperado con la solución: cada prueba usa el proveedor adecuado según lo que verifica.

Errores comunes del ejercicio

Error Causa Solución Contexto no creado Falta EnsureCreated Llamar a EnsureCreated en el constructor Conexión cerrada La conexión SQLite se cierra antes de la prueba Mantener la conexión abierta durante la prueba Datos compartidos entre pruebas Se usa la misma base de datos en todas las pruebas Usar un nombre único o una conexión por prueba InMemory no aplica restricciones El proveedor no simula SQL Usar SQLite en memoria para pruebas realistas Logger no inyectado Falta ILogger en el constructor Inyectar el logger o usar NullLogger Tests dependientes Se reutiliza el mismo contexto Crear un contexto por prueba

Reto resuelto: Test de concurrencia con SQLite en memoria

Reto: Escribir un test que verifique el comportamiento de la concurrencia optimista con SQLite en memoria. El test debe cargar la misma orden en dos contextos, modificar el cliente en uno y el estado en el otro, y verificar que el segundo SaveChanges lanza DbUpdateConcurrencyException.

Solución paso a paso:

Paso 1: Crear el test en tests/AceriaData.Tests/Sqlite/ConcurrenciaSqliteTests.cs:

```
using AceriaData.Domain.Entities;
using AceriaData.Infrastructure.Persistence;
using Microsoft.Data.Sqlite;
using Microsoft.EntityFrameworkCore;

namespace AceriaData.Tests.Sqlite;

public class ConcurrenciaSqliteTests : IDisposable
{
    private readonly SqliteConnection _connection;

    public ConcurrenciaSqliteTests()
    {
        _connection = new SqliteConnection("Data Source=:memory:");
        _connection.Open();

        using var context = CrearContexto();
        context.Database.EnsureCreated();

        context.OrdenesFabricacion.Add(new OrdenFabricacion { NumeroOrden = "OF-001", Cliente =
"Constructora del Norte", Estado = "Pendiente", FechaCreacion = DateTime.Now });
        context.SaveChanges();
    }

    private AceriaDbContext CrearContexto()
    {
        var options = new DbContextOptionsBuilder<AceriaDbContext>()
            .UseSqlite(_connection)
            .Options;
        return new AceriaDbContext(options);
    }

    [Fact]
    public void ActualizarOrden_ConDosContextos_LanzaExcepcionDeConcurrencia()
    {
        using var contextA = CrearContexto();
        using var contextB = CrearContexto();

        var ordenA = contextA.OrdenesFabricacion.First(o => o.NumeroOrden == "OF-001");
        var ordenB = contextB.OrdenesFabricacion.First(o => o.NumeroOrden == "OF-001");

        ordenA.Cliente = "Constructora del Norte Actualizada";
        contextA.SaveChanges();

        ordenB.Estado = "EnProceso";

        var excepcion = Record.Exception(() => contextB.SaveChanges());

        Assert.NotNull(excepcion);
        Assert.IsType<DbUpdateConcurrencyException>(excepcion);
    }

    public void Dispose()
    {
        _connection.Dispose();
    }
}
```

Línea 1: using AceriaData.Domain.Entities; → importa las entidades.

Línea 2: using AceriaData.Infrastructure.Persistence; → importa el DbContext.

Línea 3: using Microsoft.Data.Sqlite; → importa SQLite.

Línea 4: using Microsoft.EntityFrameworkCore; → importa EF Core.

Línea 6: namespace AceriaData.Tests.Sqlite; → declara el espacio de nombres.

Línea 8: `public class ConcurrenciaSqliteTests : IDisposable` → declara la clase de tests.

Línea 10: `private readonly SqlConnection _connection;` → campo de la conexión.

Línea 12: `public ConcurrenciaSqliteTests()` → constructor.

Línea 14: `_connection = new SqlConnection("Data Source=:memory:");` → crea la conexión.

Línea 15: `_connection.Open();` → abre la conexión.

Línea 17: `using var context = CrearContexto();` → crea el contexto.

Línea 18: `context.Database.EnsureCreated();` → crea la base de datos.

Línea 20: `context OrdenesFabricacion.Add(new OrdenFabricacion { ... });` → añade la orden.

Línea 21: `context.SaveChanges();` → guarda los cambios.

Línea 24: `private AceriaDbContext CrearContexto()` → declara el método.

Línea 26: `var options = new DbContextOptionsBuilder()` → crea las opciones.

Línea 27: `.UseSqlite(_connection)` → configura SQLite.

Línea 28: `.Options;` → obtiene las opciones.

Línea 29: `return new AceriaDbContext(options);` → devuelve el contexto.

Línea 32: `[Fact]` → atributo de test.

Línea 33: `public void ActualizarOrden_ConDosContextos_LanzaExcepcionDeConcurrencia()` → declara el test.

Línea 35: `using var contextA = CrearContexto();` → crea el contexto A.

Línea 36: `using var contextB = CrearContexto();` → crea el contexto B.

Línea 38: `var ordenA = contextA.OrdenesFabricacion.First(o => o.NumeroOrden == "OF-001");` → carga la orden en A.

Línea 39: `var ordenB = contextB.OrdenesFabricacion.First(o => o.NumeroOrden == "OF-001");` → carga la orden en B.

Línea 41: `ordenA.Cliente = "Constructora del Norte Actualizada";` → modifica el cliente en A.

Línea 42: `contextA.SaveChanges();` → guarda los cambios de A.

Línea 44: `ordenB.Estado = "EnProceso";` → modifica el estado en B.

Línea 46: `var excepcion = Record.Exception(() => contextB.SaveChanges());` → captura la excepción.

Línea 48: `Assert.NotNull(excepcion);` → comprueba que hay excepción.

Línea 49: `Assert.IsType(excepcion);` → comprueba el tipo.

Línea 52: `public void Dispose()` → declara el método Dispose.

Línea 54: `_connection.Dispose();` → libera la conexión.

| Paso 2: Ejecutar el test:

```
dotnet test tests/AceriaData.Tests/AceriaData.Tests.csproj --filter "ConcurrenciaSqliteTests"
```

Resultado esperado: el test pasa y verifica que EF Core lanza `DbUpdateConcurrencyException` cuando dos contextos modifican la misma orden.

Ampliación técnica obligatoria — estrategia de testing en cuatro niveles

La fuente original cubre xUnit, EF Core InMemory y SQLite en memoria. El contrato del curso amplía el punto para separar pruebas unitarias de Application, dobles de proveedor y pruebas reales contra SQL Server.

Paso 13: Añadir unit tests de Application con xUnit + Moq, sin EF Core

Mockea las abstracciones del repositorio, no DbSet para consultas LINQ.

```
var repo = new Mock<IOrdenRepositorio>();
repo.Setup(r => r.ObtenerPorNumero("OF-UNIT-001"))
    .Returns(new OrdenFabricacion
    {
        NumeroOrden = "OF-UNIT-001",
        Cliente = "Cliente unitario",
        Estado = "Pendiente"
    });

var sut = new ConsultarOrdenUseCase(repo.Object);
var resultado = sut.Ejecutar("OF-UNIT-001");

Assert.Equal("Cliente unitario", resultado.Cliente);
repo.Verify(r => r.ObtenerPorNumero("OF-UNIT-001"), Times.Once);
```

Este nivel no referencia Microsoft.EntityFrameworkCore desde Application.

Paso 14: Mantener InMemory y SQLite como dobles con limitaciones explícitas

- InMemory no es una base relacional y no reproduce transacciones, SQL ni semántica del proveedor real.
- SQLite en memoria permite pruebas relacionales rápidas, pero su dialecto, funciones, tipos, collation y comportamiento difieren de SQL Server.
- No uses SQLite para afirmar que el rowversion autogenerado de SQL Server funciona igual.

Paso 15: Crear una DatabaseFixture contra SQL Server LocalDB real

```
public sealed class SqlServerDatabaseFixture : IAsyncLifetime
{
    private readonly string _dbName = $"AceriaData_Test_{Guid.NewGuid():N}";
    public string ConnectionString =>
        $"Server=(localdb)\
\MSSQLLocalDB;Database={_dbName};Trusted_Connection=True;TrustServerCertificate=True";

    public async Task InitializeAsync()
    {
        var options = new DbContextOptionsBuilder<AceriaDbContext>()
            .UseSqlServer(ConnectionString)
            .Options;

        await using var context = new AceriaDbContext(options);
        await context.Database.MigrateAsync();
    }

    public async Task DisposeAsync()
    {
        var options = new DbContextOptionsBuilder<AceriaDbContext>()
            .UseSqlServer(ConnectionString)
            .Options;

        await using var context = new AceriaDbContext(options);
        await context.Database.EnsureDeletedAsync();
    }
}
```

La fixture crea una base aislada y aplica **las migraciones reales** antes de ejecutar los tests.

Paso 16: Usar Respawn para restaurar los datos entre tests

Respawn no sustituye las migraciones: se usa después de preparar el esquema.

```
await using var connection = new SqlConnection(fixture.ConnectionString);
await connection.OpenAsync();

var checkpoint = await Respawner.CreateAsync(connection, new RespawnerOptions
{
    DbAdapter = DbAdapter.SqlServer,
    TablesToIgnore = new Table[] { "__EFMigrationsHistory" }
});

await checkpoint.ResetAsync(connection);
```

Tras el reset, comprueba que el historial de migraciones se conserva y que los datos del test anterior han desaparecido.

Paso 17: Añadir un host `AceriaData.Api` mínimo y probarlo con `WebApplicationFactory<Program>`

El host API puede añadirse sin mover responsabilidades de Domain/Application/Infraestructure.

```
public class ApiIntegrationTests : IClassFixture<WebApplicationFactory<Program>>
{
    private readonly HttpClient _client;

    public ApiIntegrationTests(WebApplicationFactory<Program> factory)
    {
        _client = factory.CreateClient();
    }

    [Fact]
    public async Task GetOrdenes_DevuelveSuccess()
    {
        var response = await _client.GetAsync("/api/ordenes");
        response.EnsureSuccessStatusCode();
    }
}
```

La configuración de pruebas debe sustituir la cadena de conexión por la base LocalDB aislada de la fixture.

Paso 18: Ejecutar toda la matriz de pruebas

```
dotnet test AceriaData.sln --configuration Release
```

El CI debe fallar si falla cualquiera de estos niveles:

1. Unit: xUnit + Moq, sin EF Core.
2. Provider-double: InMemory / SQLite con limitaciones documentadas.
3. Integration DB: SQL Server LocalDB + migraciones + DatabaseFixture + Respawn.
4. Integration HTTP: WebApplicationFactory + API + base SQL Server de pruebas.

Analogía final

El testing con EF Core en una acería es como las pruebas de calidad de las planchas antes de enviarlas al cliente. El proveedor InMemory es como una prueba rápida que verifica que la plancha tiene el tamaño correcto pero no comprueba la resistencia. SQLite en memoria es como una prueba completa que verifica la resistencia, la composición y las tolerancias. InMemory y SQLite en memoria se eligen por

el comportamiento que se necesita comprobar, no por un ranking universal de velocidad. InMemory no ofrece semántica relacional; SQLite ofrece semántica relacional útil, pero sigue difiriendo de SQL Server. Las pruebas se ejecutan en cada cambio para detectar errores antes de que lleguen al cliente. Así funciona el testing con EF Core: se elige el proveedor adecuado según lo que se verifica y se ejecutan las pruebas de forma continua.

| Resultado esperado

Al final del ejercicio, deberías haber:

Creado el proyecto `AceriaData.Tests` con xUnit.

Añadido las referencias a los proyectos.

Añadido los paquetes de EF Core para pruebas.

Creado la clase base `TestBase`.

Creado los tests del repositorio de órdenes.

Creado los tests de la unidad de trabajo.

Creado los tests con InMemory.

Creado los tests con SQLite en memoria.

Ejecutado todos los tests.

Diagnosticado el error de la conexión cerrada.

Creado el test de concurrencia con SQLite en memoria.

| Conexión con el siguiente punto

En este punto se ha profundizado en el testing con EF Core, incluyendo la configuración del proyecto de pruebas, el uso del proveedor InMemory y SQLite en memoria, y la escritura de tests que verifican el comportamiento de la capa de persistencia. Se han comparado InMemory y SQLite en memoria por comportamiento, no mediante un ranking universal de velocidad. InMemory no reproduce semántica relacional; SQLite ofrece semántica relacional útil para pruebas rápidas, pero sigue teniendo diferencias de proveedor frente a SQL Server, por lo que las pruebas dependientes del proveedor se ejecutan también contra SQL Server LocalDB. En el siguiente punto se estudiarán las buenas prácticas y anti-patrones en persistencia empresarial, cerrando el Módulo 5.
